Terraform Hands-on Exam and Q&A

Exam Overview

Duration: **3-5 hours**

- **Section 1: Multiple Choice & Short Answer Questions (20 questions)** (~1 hour)
- **Section 2: Hands-on Practical Assignments** (~2-4 hours)
    - Task-based questions requiring Terraform configuration
    - Infrastructure provisioning and troubleshooting

Exam Structure

Section 1: Q&A (20 Questions)

- **Terraform Fundamentals (5 questions)**

    - What is Terraform and how does it differ from other IaC tools?

      Terraform is an open-source Infrastructure as Code (IaC) tool which allows you to define and manage your infrastructure in a declarative manner. Terraform primarily focuses on provisioning and managing infrastructure components like servers, networks, and databases. Other tools might specialize in configuration management or application deployment like Ansible and ArgoCD .

    - Explain Terraform's declarative nature and state management.

      In declarative language, you define the desired state of your infrastructure. You specify what resources you need (servers, networks, databases) and their configurations, but you don't dictate how Terraform should create or manage them. Terraform keeps track of infrastructure using state files, allowing it to know what changes are needed when running updates.

    - What is the purpose of the Terraform provider?

      The purpose of a Terraform provider is to act as a bridge between Terraform and the API of a service or platform you want to manage, such as AWS, Azure, or GCP. It effectively incorporates all the necessary SDKs and libraries for that specific service.

    - How does Terraform handle dependency resolution?

1. Implicit Dependencies: Terraform automatically detects dependencies by analyzing resource attributes. When one resource refers to another using interpolation syntax like aws_instance.example[0].public_ip.
2. Explicit Dependencies: when dependencies are not automatically detected , we can explicitly define them using the depend_on meta-argument.
3. Dependency Graph: Terraform uses the information from implicit and explicit dependencies to build a dependency graph.The graph represents the relationships between resources and determines the order in which they need to be created, updated, or destroyed.

- What are the key components of a Terraform configuration file?
  1. Providers : plugins that enable Terraform to interact with specific services or platforms.
  2. Resources : Resources represent the individual components of your infrastructure that you want to manage
  3. Data Sources: Data sources allow you to fetch information about existing infrastructure.
  4. Variables : make your configurations more flexible and reusable.
  5. Outputs : display information about your infrastructure.
  6. Modules : reusable units of Terraform code that encapsulate a set of resources and their configurations

- **State Management & Backend Configuration (3 questions)**

  - Explain the difference between `terraform refresh`, `terraform plan`, and `terraform apply`.
    1. Terraform refresh : Synchronizing your terraform state file with the actual state of the infrastructure
    2. Terraform Plan : Preview the changes before making them.
    3. Terraform Apply: Provision the resources in your environment.
  - What is the difference between local and remote backends?
    1. Local backend: The state file is stored on the local filesystem of the machine where you run Terraform commands.
    2. Remote backend: The state file is stored in a remote location (Cloud Storage)
  - How can you prevent state corruption when multiple engineers work on the same infrastructure?

    State Locking on state file : Remote backends provide state locking. When an engineer runs a Terraform command that modifies the state like apply or destroy. Terraform attempts to acquire a lock on the state file.

- **Terraform Modules & Reusability (4 questions)**

  - What are the benefits of using Terraform modules?

1. Reusability: write one time and use many. Define a piece of infrastructure once and reuse it across different projects.
2. Code duplication: Not repeating the same configuration for similar resources.
3. Organization: Breaking down complex infrastructure into smaller, modular units, you can improve the readability and maintainability of the code.

- Explain how to pass variables to a Terraform module.
  1. Calling the Module and Passing Values .
  2. Defining Variables in the Module.
  3. Using Variables within the Module.
- What is the difference between `count` and `for_each`?
  1. Count : creates a specified number of identical instances. Take an integer value that will define how many instances to create. Instances are created in a specific order.
  2. For_each : creates instances based on the elements of a set or map . Each element in the map or set represents an instance to be created . Instances identified by the keys.The order of elements in the input map or set does not affect the order of resource creation.
- How do you source a module from a Git repository?

  When calling the module and passing values in the source section we write : git::REPO_LOCATION//folder?ref=BRANCH

- **Terraform with AWS (4 questions)**
  - How do you create an EC2 instance with Terraform?

```
resource "aws_instance" "web" {

ami = data.aws_ami.ubuntu.id

instance_type = "t3.micro"

tags = { Name = "HelloWorld" }

 }
```

  - What are the required fields for defining a VPC in Terraform?
    1. Cidr_block : IP address range for instance "10.0.0.0/16" .
    2. Region : AWS region where you want to create your VPC for instance us-east-1 .
  - Explain how Terraform manages IAM policies in AWS.

Terraform uses a declarative approach to IAM Policy Management. Instead of describing how to create policy, you define what you want the policy to be. terraform will already make sure to translate this statement into AWS API commands that created the actual policy.
For instance : Bring access for all information about EC2 instances .

- How do you use Terraform to provision and attach an Elastic Load Balancer?
    1. Aws_lb resource creation : Load balancer itself.
    2. Aws_lb_targer_group resource creation : load balancer will distribute traffic to the defined target group.
    3. Aws_lb_target_group_attachment resource creation: associate EC2 instances with target group.
    4. Aws_lb_listener resource creation : specify how the load balancer accepts incoming traffic .

- **Debugging & Error Handling (4 questions)**

    - What does the `terraform validate` command do?

      Spell checker for your infrastructure code . verifies that Terraform files adhere to the correct syntax . analyzes the relationships between different resources, variables, and modules within your configuration.

    - How can you debug Terraform errors effectively?
        1. Use terraform validate , terraform plan and review carefully.
        2. Use Version Control (Git)
        3. Understand state management .
        4. Consult provider docs and search community resources .
        5. TF_LOG=”DEBUG” breaks down state file configuration .
    - What is Terraform’s `ignore_changes` lifecycle policy used for?

      The lifecycle policy tells Terraform to ignore changes made to a resource's attributes after its initial creation. For example, if you create an EC2 instance with the t2.micro instance type, and someone later changes the instance type (either manually or through another process), Terraform will not revert it back to t2.micro.

    - How do you import existing AWS infrastructure into Terraform?

      Using command terraform import with resource type and ID to bring the resource.
      terraform import aws_instance.vm i-0abcdef0123456789 .